

B1-2 : 리눅스 프로세스 및 시스템 리소스 트러블슈팅

Memory Leak · CPU Spike · Deadlock — 재현, 분석, 보고

환경 : macOS + OrbStack + Ubuntu 22.04 (agent-admin 계정)
실험 도구 : agent-leak-sim.py (Python 시뮬레이터, sudo 불필요)

미션 개요 — 목표와 3가지 장애 유형

한 줄 요약: 실제 서버 장애 3가지를 직접 재현하고, 로그를 증거로 원인을 분석하여

GitHub Issue 형태의 기술 보고서를 작성한다

핵심 전환: “장애 = 감으로 추측하는 것” → “장애 = 데이터로 증명하고 기록하는 것”

장애 유형	현상	원인
Memory Leak (OOM)	프로세스가 갑자기 종료됨	메모리를 계속 할당만 하고 해제하지 않음
CPU Spike	CPU 사용률이 급상승 후 프로세스 종료	특정 연산이 CPU를 독점
Deadlock	프로세스가 살아있으나 완전히 멈춤	두 스레드가 서로의 자원을 무한 대기

미션 완료 후 할 수 있는 것: 메모리/CPU 이상 패턴을 로그로 식별, Deadlock을 시스템 도구로 진단,
그리고 이를 GitHub Issue 형태의 기술 리포트로 작성하여 팀과 공유

환경 설정 ① — 기본 환경 & B1-2 추가 환경변수

실행 환경: macOS + OrbStack + Ubuntu 22.04 (b1-lab VM, agent-admin 계정)

B1-1 기반 환경 재사용: AGENT_HOME / AGENT_PORT 환경변수, /home/agent-admin/agent-app/ 디렉토리,
monitor.sh (매분 자동 실행되는 리소스 모니터)

환경 복구 절차: `chmod +x 'b1-1 setup.sh' → sudo bash 'b1-1 setup.sh'`

환경변수 (.bashrc)	기본값	의미 / 허용 범위
MEMORY_LIMIT	256	메모리 상한선 (MB, 50~512)
CPU_MAX_OCCUPY	50	CPU 최대 점유율 (% , 10~100)
MULTI_THREAD_ENABLE	true	멀티스레드(Deadlock 시나리오) 사용 여부 (true/false)

적용 방법: `.bashrc`에 `export` 구문 추가 → `source ~/.bashrc` → `echo $VAR`로 값 확인

환경 설정 ② — 원본 바이너리 대안: agent-leak-sim.py

문제: 원본 agent-leak-app은 Linux 바이너리이며, 실습 PC는 sudo 권한이 제한되어

/var/log/agent-app/ 같은 시스템 경로에 자유롭게 쓸 수 없음

해결: 동일한 로그 패턴(형식·문구)을 그대로 출력하는 Python 시뮬레이터를 직접 제작

실제 메모리 할당(bytearray 25MB), 실제 CPU 점유(math.sqrt busy-loop),

실제 threading.Lock 기반 Deadlock 구현 — 모두 sudo 없이 동작

로그 경로 자동 결정: /var/log/agent-app/ 쓰기 가능하면 사용, 아니면 ./logs/ 로 대체

케이스	MEMORY_LIMIT	CPU_MAX_OCCUPY	MULTI_THREAD_ENABLE
1 — Memory Leak	256 MB	100 %	false
2 — CPU Spike	9999 MB	50 %	false
3 — Deadlock	9999 MB	100 %	true

실행: python3 agent-leak-sim.py → 메뉴에서 1/2/3 선택 시 위 프리셋이 자동 적용됨

Case 1 — Memory Leak: 현상 & 증거

현상: 메모리가 25MB씩 지속 증가하여 MEMORY_LIMIT(256MB)을 초과

조건: MEMORY_LIMIT=256 / CPU_MAX_OCCUPY=100 / MULTI_THREAD_ENABLE=false

[agent_app.log 발췌]

09:34:48 [INFO] [Memory] Increasing... (+25 MB) Total: 25 MB

09:34:50 [INFO] [Memory] Increasing... (+25 MB) Total: 50 MB

...

09:35:32 [INFO] [Memory] Increasing... (+25 MB) Total: 250 MB

09:35:38 [INFO] [Memory] Increasing... (+25 MB) Total: 275 MB <- 한계(256MB) 초과

[CRITICAL] [MemoryGuard] Memory limit exceeded (275MB >= 256MB)

[CRITICAL] [MemoryGuard] Self-terminating process XXXX to prevent system instability.

>>> [SYSTEM] SELF-TERMINATED (Memory Limit Exceeded) <<<

메모리 상승 속도: 0MB → 275MB 약 50초, +25MB/단계로 일정하게 증가

Case 1 — Memory Leak: 원인 분석 & 조치

앱 내부에서 메모리를 할당 후 해제하지 않는 메모리 누수(Memory Leak) 발생

25MB 단위로 계속 누적 → MEMORY_LIMIT(256MB) 초과 시 MemoryGuard가 SIGKILL로 자가 종료

정상 동작: 할당 → 사용 → 해제 (일정 유지) | 누수 동작: 할당 → 사용 → [해제 안 함] → 계속 증가

구분	설정	결과
Before	MEMORY_LIMIT = 256 MB	약 50초 후 한계 도달 → SELF-TERMINATED
After	MEMORY_LIMIT = 512 MB	더 오랜 시간 생존 확인 (한계치 상향만으로는 결국 재발)

✅ 근본 해결: 소스코드에서 사용 후 불필요한 데이터를 주기적으로 해제(del / free)하는 리팩토링

Case 2 — CPU Spike: 현상 & 증거

현상: CPU 레벨이 1→10까지 램프업, Lv≥8 구간에서 실제 CPU 사용률 99.7% > 50%(임계치)

조건: CPU_MAX_OCCUPY=50 / MEMORY_LIMIT=9999 / MULTI_THREAD_ENABLE=false

[agent_app.log 발췌]

```
17:37:50 [INFO] --- Step Info: Mode=UP, CPU Lv=8, Mem=200MB ---  
17:37:50 [INFO] [CPU] Occupy core for 4.0s (Level 8)  
17:37:50 [WARNING] [Watchdog] CPU usage spike detected: 99.7% > 50%  
17:37:50 [CRITICAL] [Watchdog] INITIATING EMERGENCY ABORT (SIGTERM)  
>>> [SYSTEM] WATCHDOG: PROCESS TERMINATED <<<
```

CPU Lv=10 도달 패턴이 약 1분 42초 주기로 반복 관측됨 — 아래 4회 관측 시각으로 증명

발생 시각 (agent_app.log)	CPU Lv	Mem	이전 관측 대비 간격
15:38:19	10	225MB	—
15:40:01	10	225MB	1분 42초 (102초)
15:41:43	10	225MB	1분 42초 (102초)
15:43:26	10	225MB	1분 43초 (103초)

Case 2 — CPU Spike: 원인 분석 & 조치

앱 내부 CPU 점유 로직이 레벨(1~10)에 따라 코어를 독점 — Lv=10에서는 5초간 코어 완전 점유
약 1분 42초 주기로 CPU_MAX_OCCUPY(50%) 초과 → Watchdog이 SIGTERM으로 프로세스 종료

증거: "Lv=10 → 5초간 코어 완전 점유"는 아래 코드 로직과 실측 로그로 확인됨

```
[agent-leak-sim.py — cpu_step(level) 핵심 로직]
burn_sec = level * 0.5    # Lv=10 → 10 × 0.5 = 5.0초 (코어 점유 시간)
_burn(burn_sec)           # 5.0초 동안 sqrt 연산 반복 (busy-loop, CPU 점유)
actual = (CPU시간 증가량 / wall시간) × 100 # 실측 점유율(%)
```

[실측 로그 — Lv=8(burn_sec=4.0s) 동일 로직 적용 결과]
17:37:50 [INFO] [CPU] Occupy core for 4.0s (Level 8)
17:37:50 [WARNING] [Watchdog] CPU usage spike detected: 99.7% > 50%
→ wall ≈ 4.0s, actual ≈ 99.7% (거의 100% 코어 점유 확인)
→ 동일 busy-loop이 Lv=10(5.0s)에도 적용 → 5.0초간 ≈ 100% 점유

구분	설정	결과
Before	CPU_MAX_OCCUPY = 50 %	Lv=10 도달 시 99.7% > 50% → 즉시 WATCHDOG 종료
After	CPU_MAX_OCCUPY = 80~90 %	더 높은 사용을 허용 → 안정적 동작 유지

✓ 근본 해결: CPU를 독점하는 연산을 여러 작업으로 분산하거나, 중간에 sleep/yield를 삽입하여 다른 프로세스에 양보

Case 3 — Deadlock: 현상 & 증거

현상: 프로세스가 종료되지 않고 PID 유지, CPU/MEM 변화 없이 로그가 완전히 정지

조건: MULTI_THREAD_ENABLE=true / MEMORY_LIMIT=9999 / CPU_MAX_OCCUPY=100

[agent_app.log — 마지막 로그]

17:45:28 [INFO] [Thread-A] Acquired Lock-1, WAITING for Lock-2...

17:45:28 [INFO] [Thread-B] Acquired Lock-2, WAITING for Lock-1...

(이후 로그 없음 — 영원히 멈춤)

[ps -ef | grep agent-leak-sim]

PID 28648 TIME 0:55.09 Python agent-leak-sim.py

-> PID 존재(생존) + TIME 값 변화 없음(작업 정지) = Deadlock

A는 B가 가진 Lock-2를, B는 A가 가진 Lock-1을 서로 기다리며 영원히 대기 = Deadlock

Case 3 — Deadlock: 원인 분석 & 조치

Deadlock 4대 조건	우리 실습에서의 발생 양상
상호 배제 (Mutual Exclusion)	threading.Lock() — 한 번에 한 스레드만 획득 가능
점유 대기 (Hold and Wait)	Thread-A가 Lock-1을 쥔 채 Lock-2를 대기
비선점 (No Preemption)	Thread-B가 보유한 Lock-2를 A가 강제로 가져올 수 없음
순환 대기 (Circular Wait)	A→Lock-2 대기, B→Lock-1 대기 (서로 순환 대기)

구분	설정	결과
Before	MULTI_THREAD_ENABLE = true	교착상태 발생 → PID 생존, 로그/CPU 정지(무응답)
After	MULTI_THREAD_ENABLE = false	싱글스레드 → 자원 경쟁 없음 → 정상 동작 확인

 **근본 해결: 락 획득 순서를 통일(모든 스레드가 Lock-1→Lock-2 순서로만 획득)하거나,**
 lock.acquire(timeout=...)으로 타임아웃을 설정하여 일정 시간 후 대기기를 포기하도록 개선

설명 자료 Q&A ① — 메모리 / CPU

Q. 메모리 누수가 시스템에 미치는 영향은?

A. 메모리는 OS 전체가 공유하는 자원. 한 프로세스가 계속 독점하면 다른 프로세스가 메모리를 할당받지 못해 시스템이 느려지거나 OOM 상태가 되고, 심하면 OS의 OOM Killer가 강제 종료시킴

Q. MEMORY_LIMIT을 높이는 것이 근본 해결책인가?

A. 아니다 — 임시방편. 메모리 누수(해제 안 함) 자체는 그대로이므로, 한계치를 올려도 결국 그 한계마저 초과하여 다시 종료됨. 코드에서 사용 후 메모리를 해제해야 근본 해결

Q. CPU 과점유가 시스템 전체에 미치는 영향은?

A. CPU는 여러 프로세스가 시분할(time-sharing)로 나눠 쓰는 자원. 하나가 독점하면 다른 프로세스들의 응답이 지연되고, 웹 서버라면 모든 요청이 느려져 타임아웃 발생

Q. Watchdog이 SIGTERM을 보내는 이유는?

A. SIGTERM은 “정상적으로 마무리하고 종료하라”는 신호. 파일 닫기, 로그 저장 등 마무리 작업 후 종료되어 데이터 손실이 없음 (반면 SIGKILL은 즉시 강제 종료, 최후의 수단)

설명 자료 Q&A ② — Deadlock / 리포팅

Q. Deadlock 프로세스를 어떻게 식별하나요 ?

A. PID는 있는데 CPU/MEM 변화가 없고 로그도 멈췄다면 교착상태를 의심. top -H로 모든 스레드 상태가 대기(S, D)인지 확인하고, 마지막 로그에서 WAITING / BLOCKED 패턴을 찾음

Q. MULTI_THREAD_ENABLE=false가 근본 해결책인가요 ?

A. 아니다 — 멀티스레드를 끄면 Deadlock은 없어지지만 동시 처리 성능 이점도 사라짐. 근본 해결은 락 획득 순서 통일 또는 lock.acquire(timeout=...) 적용

Q. GitHub Issue 형태로 리포트를 작성하는 이유는 ?

A. (1) 재현 방법 공유 — 같은 환경변수로 같은 문제 재현 가능, (2) Before/After 검증 기록 — 조치 효과를 누구나 확인 가능, (3) 유사 장애 시 참고 자산 — “내 머릿속 지식”을 “팀 자산”으로 전환

제출 체크리스트 — 필수 포함 항목 점검

필수 항목	OOM (Case 1)	CPU (Case 2)	Deadlock (Case 3)
1. 현상 설명 (Description)	✓	✓	✓
2. 로그 발췌 (monitor.log / agent_app.log)	✓	✓	✓
3. 앱 실행 로그 발췌	✓	✓	✓
4. ps / top 출력 캡처	✓	✓	✓
5. 근본 원인 분석 (Root Cause)	✓	✓	✓
6. Before & After 비교 (Workaround)	✓	✓	✓

생성된 결과물:

agent-leak-sim.py — 3가지 장애를 sudo 없이 재현하는 Python 시뮬레이터

report-case1-memory-leak.md / report-case2-cpu-spike.md / report-case3-deadlock.md

위 3개 보고서를 GitHub Issue로 등록 (linksat1/AI-SW-Basic) → 최종 제출

전체 흐름 요약 & 마무리

- ① agent-leak-sim.py 실행 → 케이스 선택 (1: Memory Leak / 2: CPU Spike / 3: Deadlock)
- ② 각 케이스 실행 → agent_app.log + ps/top 캡처로 증거 수집
- ③ 로그 기반 원인 분석 → 환경변수 조정 후 Before/After 비교로 효과 검증
- ④ GitHub Issue 3건 작성 (Description / Evidence / Root Cause / Workaround)
- ⑤ 최종 제출 (GitHub Repository 링크 또는 Issue 3건)

“장애 = 감으로 추측하는 것” → “장애 = 데이터로 증명하고 기록하는 것” 으로의 전환
완료